

Razor Pages Custom Tag Helpers

<https://www.learnrazorpages.com/advanced/custom-tag-helpers>

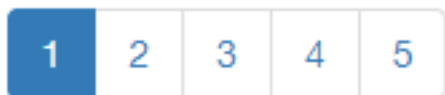
Creating Custom Tag Helpers

A wide range of Tag helpers is included as part of the Razor Pages framework. It is also possible to create your own custom tag helpers to automate the generation of HTML in a limitless range of scenarios.

The first example features a tag helper that generates paging links. The tag helper will generate a div enclosing an unordered list, with each list item having a link to a page of data. The resulting HTML should look like this:

```
<div>
  <ul class="pagination">
    <li class="active"><a href="/page?page=1" title="Click to go to page 1">1</a></li>
    <li class=""><a href="/page?page=2" title="Click to go to page 2">2</a></li>
    <li class=""><a href="/page?page=3" title="Click to go to page 3">3</a></li>
    <li class=""><a href="/page?page=4" title="Click to go to page 4">4</a></li>
    <li class=""><a href="/page?page=5" title="Click to go to page 5">5</a></li>
  </ul>
</div>
```

The CSS classes that have been applied are from the Bootstrap styles, so the HTML is rendered in the browser like this:



Two approaches to authoring the tag helper are illustrated. The first approach is based on parsing the values applied to the tag helper's attributes. The second approach will show how to bind values from attributes to properties, and is the recommended approach. The code for the first approach is shown in its entirety followed by an explanation for how it works.

Attribute Parsing

```
using Microsoft.AspNetCore.Razor.TagHelpers;
using System.Text;

namespace LearnRazorPages.TagHelpers
{
    [HtmlTargetElement("pager", Attributes = "total-pages, current-page, link-url")]
    public class PagerTagHelper : TagHelper
    {
        public override void Process(TagHelperContext context, TagHelperOutput output)
        {
            if (
                int.TryParse(context.AllAttributes["total-pages"].Value.ToString(), out int
totalPages) &&
                int.TryParse(context.AllAttributes["current-page"].Value.ToString(), out int
currentPage))
            {
                var url = context.AllAttributes["link-url"].Value;
                output.TagName = "div";
                output.PreContent.SetHtmlContent(@"<ul class=""pagination"">");

                for (var i = 1; i <= totalPages; i++)
                {
                    output.Content.AppendHtml($@"<li class=""{(i == currentPage ? "active" :
""")}""><a href=""{url}?page={i}"" title=""Click to go to page {i}"">{ i}</a></li>");
                }
                output.PostContent.SetHtmlContent("</ul>");
                output.Attributes.Clear();
            }
        }
    }
}
```

Tag helpers inherit from the abstract TagHelper class which defines a couple of virtual methods: Process and ProcessAsync.

These methods are where the HTML output is generated. The vast majority of helpers implement the synchronous Process method. The async version should

be used in cases where generating the tag helper's output requires communicating with "out of process" resources, such as files, databases, web services and so on. Both versions of the Process method take two parameters, a TagHelperContext object and a TagHelperOutput object. The TagHelperContext object contains information about the current tag being operated on including all of its attributes. The TagHelperOutput object represents the output generated by the tag helper. As the Razor parser encounters an element in a page that is associated with a tag helper, the tag helper's ProcessAsync method is invoked and generates output accordingly.

Associating a tag with a TagHelper

You are encouraged to name your tag helper class with the "TagHelper" suffix. In this example, the tag helper class has been named `PagerTagHelper`. By convention, the tag helper will target elements that have the same name as the class up to the suffix - in this case: `<pager>`. If you want to ignore the suffix convention and/or target an element with a different name, you must use the `HtmlTargetElement` attribute to specify the name of the tag that your helper should process.

You can further refine which elements to target via the `Attributes` parameter of the `HtmlTargetElement` attribute. In the example above, three attributes are passed to the `Attributes` parameter: `current-page`, `total-pages` and `link-url`. The fact that they have been specified as parameters makes them mandatory, so this helper will only act on `<pager>` elements that have all three attributes declared. Since there is a match between the target element and the tag helper name in this example, it might seem superfluous to pass "pager" to the `HtmlTargetElement` attribute, but if it is omitted, an overload of the attribute is used which has the `Tag` property preset to a wildcard (*). In other words, omitting the tag name but passing a list of required attributes will result in the tag helper acting upon *any* element that features all of the required attributes. If you wanted to target a selection of elements, you can set multiple `HtmlTargetElement` attributes.

Generating HTML

The `TagHelperContext.Attributes` collection is queried for values via their string-based index. Further processing for this tag helper is undertaken only if the `totalPages` and `currentPage` values can be parsed as numbers.

The `TagName` property of the `TagHelperOutput` parameter is set to `div`. This will result in `pager` being replaced by `div` in the final output. This is needed otherwise the tag will retain the name of `pager` when it is converted to HTML and as a consequence will not be rendered by the browser.

The `Process` method's output parameter has (among others) the following properties: `PreContent`, `Content` and `PostContent`. `PreContent` appears after the opening tag specified by the `TagName` property and before whatever is applied to the `Content` property. `PostContent` appears after the `Content`, and before the closing tag specified by the `TagName` property



Each of these properties have `SetHtmlContent` and `AppendHtml` methods that enables content to be set or built. In this example, the `Pre-` and `PostContent` properties are set to an opening and closing `<ul>` tag. The list items are generated in a loop and appended to the `Content` property via the `AppendHtml` method. Finally, the `Attributes` collection's `Clear()` method is used to remove all of the custom attributes (`total-pages`, `link-url` etc) from the output. If you do not remove the custom attributes, they will be rendered in the final HTML.

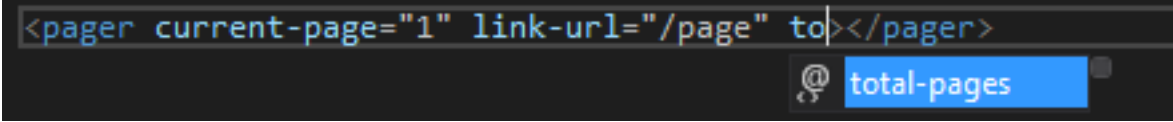
Finally, you enable the tag helper in the `_ViewImports.cshtml` file with the `@addTagHelper` directive:

```
@addTagHelper *, LearnRazorPages
```

When you use a tag helper that relies on attribute parsing, the tag helper itself appears in a different colour in the page than other HTML tags, but the attributes are presented the same as any other attribute for an HTML tag:

```
<pager current-page="1" link-url="/page" total-pages="5"></pager>
```

However, they are still available to Intellisense:



```
<pager current-page="1" link-url="/page" to|></pager>
```

The screenshot shows an IDE with a dark theme. The text in the editor is `<pager current-page="1" link-url="/page" to|></pager>`. The `to|` part is highlighted with a blue background. Below the text, there is a small icon of a person with a speech bubble, and to its right, a blue box containing the text `total-pages`. This indicates that the IDE's Intellisense feature is suggesting the `total-pages` attribute for the `pager` tag helper.

No constraint has been placed on the attributes in terms of the types of acceptable values. It is perfectly possible for the consumer of the tag helper to provide non-integer values, which is why the check for data type is necessary in the Process method above. You can solve this by binding attribute values to properties instead.

Binding to simple properties

Here's an example of the Pager tag helper modified to work with simple properties:

```
using Microsoft.AspNetCore.Razor.TagHelpers;
using System.Text;

namespace LearnRazorPages.TagHelpers
{
    [HtmlTargetElement("pager", Attributes = "total-pages, current-page, link-url")]
    public class PagerTagHelper : TagHelper
    {
        public int CurrentPage { get; set; }
        public int TotalPages { get; set; }
        [HtmlAttributeName("link-url")]
        public string Url { get; set; }

        public override void Process(TagHelperContext context, TagHelperOutput output)
        {
            output.TagName = "div";
            output.PreContent.SetHtmlContent(@"<ul class=""pagination"">");

            for (var i = 1; i <= TotalPages; i++)
            {
                output.Content.AppendHtml($@"<li class=""{(i == CurrentPage ? "active" : ""})""><a href=""{Url}?page={i}"" title=""Click to go to page {i}"">{i}</a></li>");
            }
            output.PostContent.SetHtmlContent("</ul>");
            output.Attributes.Clear();
        }
    }
}
```

The main body of the Process method is almost identical to the previous example, except that it now works on the properties of the PagerTagHelper class. The code for extracting the attribute values has been removed. It is no longer required as the tag helper will take care of generating attributes for each

public property and binding incoming values from the attributes to the property. By default, attribute names are generated as lower-case versions of the property name. If a property name includes capital letters after the first character, a hyphen is inserted, so the `CurrentPage` property becomes a `current-page` attribute.

Sometimes you might want to map a property name to a different attribute name. This is achieved by decorating the property with the `HtmlAttributeName` attribute, passing in the name of the attribute that should be generated for the property. You can see this above where the `Url` property is mapped to an attribute named `link-url`.

The generated HTML is identical to the first example, but the chief difference is that the tag helper is now strongly-typed, and it will be a design time error to pass an incorrectly typed value to an attribute - as can be seen here where a non-integer value has been supplied to the `total-pages` attribute in the page:

```
<pager current-page="1" link-url="/page" total-pages="foo"></pager>
```

You also get the benefit of Intellisense telling you what type of value is expected as well as if it is required:

```
<pager current-page="1" link-url="/page" total-pages=""></pager>
```

Attribute 'total-pages' on tag helper element 'pager' requires a value. Tag helper bound attributes of type 'System.Int32' cannot be empty or contain only whitespace.

 `int RazorPagesTest.TagHelpers.PagerTagHelper.TotalPages`

Finally, you can also benefit from XML comments on properties which can aid users of your tag helpers:

```
public class PagerTagHelper : TagHelper
{
    public int CurrentPage { get; set; }
    public int TotalPages { get; set; }
    /// <summary>
    /// The url that the paging link should point to
    /// </summary>
    [HtmlAttributeName("link-url")]
    public string Url { get; set; }
```

```
public override void Process(TagHelperContext context, TagHelperOutput output)
```

```
<pager current-page="1" link-url="/page" total-pages="5"></pager>
```

 `string` RazorPagesTest.TagHelpers.PagerTagHelper.Url
The url that the paging link should point to

Binding to complex properties

The final example illustrates how to bind the tag helper to a complex property. This tag helper outputs company details using what Google refers to as **Rich Snippets** - additional attributes added to HTML to provide structure to content.

The complex property that the tag helper will bind to is represented by the following Organisation class:

```
public class Organisation
{
    public string Name { get; set; }
    public string StreetAddress { get; set; }
    public string AddressLocality { get; set; }
    public string AddressRegion { get; set; }
    public string PostalCode { get; set; }
}
```

It is declared as a property of the TagHelper class:

```
using Microsoft.AspNetCore.Razor.TagHelpers;

namespace LearnRazorPages.TagHelpers
{
    public class CompanyTagHelper : TagHelper
    {
        public Organisation Organisation { get; set; }

        public override void Process(TagHelperContext context, TagHelperOutput output)
        {
            output.TagName = "div";
            output.Attributes.Add("itemscope itemType", "http://schema.org/Organization");

            output.Content.SetHtmlContent(
                $"@<span itemprop=""name"">{Organisation.Name}</span>
                <address itemprop=""address"" itemscope
itemtype=""http://schema.org/PostalAddress"">
                <span itemprop=""streetAddress"">{Organisation.StreetAddress}</span><br>
```

```

        <span
itemprop=""addressLocality"">{Organisation.AddressLocality}</span><br>
        <span itemprop=""addressRegion"">{Organisation.AddressRegion}</span>
        <span itemprop=""postalCode"">{Organisation.PostalCode}</span>");
    }
}
}
1.

```

An Organisation property named Company is added to the PageModel and an instance is instantiated in the OnGet() handler of the page:

```

public class ContactModel : PageModel
{
    public string Message { get; set; }
    public Organisation Company { get; set; }

    public void OnGet()
    {
        Message = "Contact Details";
        Company = new Organisation
        {
            Name = "Microsoft Corp",
            StreetAddress = "One Microsoft Way",
            AddressLocality = "Redmond",
            AddressRegion = "WA",
            PostalCode = "98052-6399"
        };
    }
}

```

The tag helper is included in the page and the Company property of the page model is passed to the organisation attribute that has been generated by the tag helper:

```

1.    <company organisation="Model.Company"></company>

```

And this is how the generated HTML looks:

```

<div itemscope itemtype="http://schema.org/Organization">

```

```
<span itemprop="name">Microsoft Corp</span>
<address itemprop="address" itemscope
itemtype="http://schema.org/PostalAddress">
  <span itemprop="streetAddress">One Microsoft Way</span><br>
  <span itemprop="addressLocality">Redmond</span><br>
  <span itemprop="addressRegion">WA</span>
  <span itemprop="postalCode">98052-6399</span>
</address>
</div>
```

The complex property offers a more streamlined way to work in a strongly typed manner with tag helpers as it relieves the need to specify a list of properties.